

# London Bus Map — RouteManagement Adaptation Plan

## 1. Objective

Adapt the existing `RouteManagement` hiking-path application into a London bus-network editor and public viewer.

The new application should display a simplified bus-path network over an OpenStreetMap basemap. Physical path geometry must be stored only once, even when several bus routes use the same segment.

The initial application must support:

- creating bus routes manually from scratch;
- starting a new route at an arbitrary map location;
- drawing route geometry with shape points;
- connecting new geometry to existing nodes;
- connecting new geometry to an existing segment by splitting that segment;
- assigning one or more bus routes to shared segments;
- clicking a segment to see which routes use it;
- selecting a route to highlight every segment assigned to that route;
- direction-specific route membership;
- a read-only public viewer;
- a protected editor;
- staged editing, Undo, Cancel, and atomic Save.

The project should reuse the existing RouteManagement topology, geometry-editing, edit-session, SQLite, Leaflet, and repository foundations rather than being rewritten from scratch.

## 2. Existing project assessment

The existing project already has the most important architectural foundations:

- junctions as meaningful graph nodes;
- path segments as reusable graph edges;
- shape points as non-topological geometry;
- SQLite persistence;
- an SQLite R-tree spatial index;
- Leaflet rendering;
- staged edit sessions;
- replayable edit operations;
- temporary IDs;
- saved, added, replaced, and deleted object states;
- segment splitting;
- junction movement;
- segment deletion;
- degree-two junction merging;

- Undo, Cancel, and atomic Save;
- stale-session detection through a network revision.

These concepts and operations are described in the existing architecture and implementation plans.

The core graph model should remain intact:

```
Node
  ↓
Segment
  ↓
Shape points
```

The current names `junction` and `path_segment` may remain in backend code to reduce unnecessary churn. The UI may call them `nodes` and `segments` if that language feels more natural for the bus application.

Do not perform a broad rename unless it materially improves clarity.

### 3. Main simplification

Remove or disable the hiking-specific GPX import workflow from the bus application.

The bus application does not initially need:

- GPX parsing;
- GPX upload;
- trace-leg processing;
- automatic crossing detection during import;
- approximate overlap matching;
- overlap review;
- GPX source sections;
- GPX duplicate reconciliation;
- import diagnostics;
- trace-to-network conversion.

The existing overlap-matching implementation is substantial and should not be deleted carelessly if the same repository may continue supporting the hiking application. Prefer one of these structures:

#### Preferred

Create a bus-specific application mode or fork within the same repository while preserving reusable modules:

```
shared/
  geometry
  repository
  edit_session_core
  network_models
```

```
hiking/  
  gpx  
  import_draft  
  overlap_matching  
  hiking_web  
  
bus/  
  bus_routes  
  bus_editor  
  bus_web
```

### Acceptable for a quick prototype

Fork the repository into a separate London Bus Map project and remove hiking-only modules there.

Do not attempt to reuse overlap matching for ordinary bus-route drawing. The bus editor should rely on explicit user intent rather than automatic geometry inference.

## 4. Product terminology

Use the following product concepts.

### Node

A topological point at which segments begin or end.

Nodes are created when:

- a route begins;
- a route ends;
- the user explicitly ends one segment and begins another;
- new geometry connects to an existing node;
- an existing segment is split;
- several segments meet.

A node with more than two incident segments is displayed as an interchange.

Route endpoints remain nodes even when their degree is one.

### Segment

Reusable geometry between two nodes.

A segment contains:

- a start node;
- an end node;
- zero or more shape points;
- a direction setting;

- zero or more bus-route memberships.

The complete segment geometry includes its endpoint coordinates.

## Shape point

An intermediate coordinate used only to shape a segment.

Shape points are not graph nodes and must not create route connections.

## Bus route

A named or numbered London bus service, for example:

```
73
159
N73
```

A route does not initially store a mandatory ordered end-to-end traversal.

Selecting a route highlights all segments that have membership in that route.

The editor user is responsible for ensuring that the full set of tagged segments represents the intended route.

## Route direction

Each route may have two directional variants.

Direction display names should be derived from the endpoints:

```
Northbound
Southbound
Eastbound
Westbound
```

The direction name must be calculated dynamically from the start and end node coordinates.

Use the bearing from the route-direction start endpoint to its end endpoint:

```
315°-45°   Northbound
45°-135°   Eastbound
135°-225°  Southbound
225°-315°  Westbound
```

Store an optional custom direction label for future exceptional cases, but endpoint-derived naming is the default.

## 5. Data model

Retain the existing `junctions` and `path_segments` tables where practical.

Add the following tables.

### `bus_routes`

```
id
route_code
display_name
colour
created_at
updated_at
```

Requirements:

- `route_code` is unique.
- Examples: `73`, `159`, `N73`.
- `display_name` is optional.
- `colour` is optional and may be used for highlighting or badges.

### `route_directions`

```
id
bus_route_id
start_junction_id
end_junction_id
custom_direction_name
created_at
updated_at
```

Requirements:

- each direction belongs to one bus route;
- start and end nodes are required once the direction has been defined;
- derived direction name is not permanently duplicated in the database;
- the API should return the derived display name;
- a bus route may temporarily have only one defined direction.

### `route_direction_segments`

```
id
route_direction_id
path_segment_id
traversal
created_at
```

Allowed `traversal` values:

```
both
start_to_end
end_to_start
```

Add a unique constraint on:

```
route_direction_id
path_segment_id
```

This is a many-to-many relationship.

Do not store route IDs as comma-separated text or arbitrary JSON inside `path_segments`.

## Segment direction

Add a segment-level field if one does not already exist:

```
direction_mode
```

Allowed values:

```
bidirectional
start_to_end
end_to_start
```

Default:

```
bidirectional
```

Interpretation:

- the physical segment is bidirectional unless explicitly restricted;
- route-direction membership may still record how a specific route traverses it;
- the editor should prevent a route membership that contradicts a segment-level one-way restriction.

For the first implementation, contradiction may produce a clear validation error rather than complex automatic correction.

## 6. Route highlighting model

Do not require an ordered route-step model for the first release.

When a viewer selects a bus route:

```
find every route direction for that route
find every segment membership for those directions
highlight the union of those segments
```

When a viewer selects one route direction:

```
find every membership for that route direction
highlight only those segments
```

Disconnected memberships are allowed.

The application may show a warning in the editor if a route direction contains disconnected segment groups, but must not block saving in the first release.

## 7. Segment-splitting behaviour

Reuse the existing `split_path_segment` operation.

When a segment is split:

1. Create the new node.
2. Replace the source segment with two child segments.
3. Copy all ordinary segment metadata to both child segments.
4. Copy every `route_direction_segments` relationship from the source segment to both child segments.
5. Translate traversal consistently onto each child.
6. Preserve the source segment's direction mode.
7. Mark the original saved segment as replaced.
8. Perform the complete change atomically during Save.

Example:

Original:

A ————— B

Memberships:

73 Northbound, start\_to\_end

73 Southbound, end\_to\_start

476 Northbound, start\_to\_end

Split at C:

A — C — B

Result:

```
A-C
73 Northbound, start_to_end
73 Southbound, end_to_start
476 Northbound, start_to_end

C-B
73 Northbound, start_to_end
73 Southbound, end_to_start
476 Northbound, start_to_end
```

The split must remain one edit-session operation from the user's perspective.

The existing project already preserves metadata and geometry through split operations; extend that inheritance to route memberships.

## 8. Primary new workflow: draw a route from nowhere

The largest missing capability is starting new network geometry at an arbitrary map point.

Implement a new high-level edit operation and browser mode.

Suggested operation name:

```
draw_route_path
```

Alternatively, use a series of lower-level `add_node` and `add_path_segment` operations if replay remains deterministic and one complete drawing action can still be undone cleanly.

### Entry conditions

The user:

1. creates or selects a bus route;
2. selects a route direction;
3. activates `Draw route`;
4. clicks the map.

### First click

When the first click is on empty map space:

- create a temporary start node;
- begin an in-progress segment;
- associate the drawing session with the active route direction.



When the first click is on an existing node:

- reuse that node;
- begin drawing from it.

When the first click is on an existing segment:

- project the click onto that segment;
- split the segment unless the projected point is within the existing endpoint tolerance;
- reuse the new or existing endpoint node as the route start.

## While drawing

Clicking empty map space adds a shape point.

Clicking an existing node:

- completes the current segment at that node;
- automatically assigns the segment to the active route direction;
- keeps the drawing tool active and allows the user to continue from the same node.

Clicking an existing segment:

- projects the click onto the selected segment;
- reuses a nearby endpoint node when within tolerance;
- otherwise splits the existing segment;
- completes the new segment at the resulting node;
- assigns the new segment to the active route direction;
- optionally continues drawing from that node.

Double-clicking empty map space, or pressing a visible `Finish at new node` command:

- creates a new end node;
- completes the current segment;
- assigns it to the active route direction;
- ends the current drawing chain.

## Keyboard behaviour

Use:

Backspace

Remove the most recently added shape point.

Enter

Finish when the current endpoint is valid.

Esc

Cancel the active unfinished segment.

If multiple completed segments were created as part of one route-drawing action, cancelling only the currently unfinished segment should not silently remove earlier completed staged segments.

The user can use Undo to remove completed staged work.

## Shape-point versus node distinction

Default behaviour:

single click empty map  
→ shape point

double-click empty map  
→ finish at a new node

Also provide an explicit sidebar button:

Finish at new node

Do not rely solely on double-click because it is less discoverable and can be awkward on touch devices.

## Duplicate and invalid geometry checks

Reject or warn when:

- the segment has fewer than two distinct coordinates;
- the endpoint equals the start node unless explicit loop creation is enabled;
- an equivalent segment already exists between the same nodes with equivalent geometry;
- the segment length is below a small configured minimum;
- the new segment would contradict a one-way segment rule;
- a save would violate endpoint geometry invariants.

## 9. Reusing an existing segment while drawing

The user must be able to continue a bus route over geometry that already exists without drawing a duplicate line.

Add a route-assignment mode within route drawing.

Suggested interaction:

1. User reaches an existing node.
2. User clicks an incident existing segment.

3. The clicked segment receives membership in the active route direction.
4. The drawing cursor advances to the segment's opposite endpoint.
5. The user can continue selecting connected existing segments.
6. Clicking empty space begins drawing new geometry from the current node.

This is effectively a route brush with connectivity awareness.

Also support a simpler independent mode:

Paint route membership

In that mode:

- clicking a segment adds it to the active route direction;
- clicking it again removes it;
- the operation should respect traversal direction;
- the UI should indicate the currently assumed traversal direction;
- adding disconnected segments is allowed but may show a warning.

## 10. Editor modes

Replace the hiking-oriented mode structure with a smaller bus-focused editor.

### Browse

- select a segment;
- select a node;
- inspect route memberships;
- inspect degree and direction.

### Draw route

- select active route;
- select active route direction;
- create new geometry;
- begin from empty map, existing node, or existing segment;
- reuse existing segments;
- automatically assign newly created segments.

### Edit network

- draw a standalone segment;
- split a segment;
- move a node;
- delete a segment;
- remove a degree-two node and merge;
- edit shape points.

### Assign routes

- add or remove route memberships;

- choose route direction;
- set traversal;
- show disconnected membership warnings.

## Manage routes

- create route;
- edit route code and display data;
- define or change route-direction endpoints;
- view dynamically derived direction names;
- delete an unused route.

The existing Route Atlas design already treats the map as the primary workspace and the sidebar as a contextual inspector. Retain that interaction philosophy.

## 11. Segment and route labels

Implement labels as an optional feature flag.

Initial behaviour:

- labels are off by default;
- labels may appear only above a configured zoom level;
- labels show route codes attached to the segment;
- several route codes should be compactly formatted.

Example:

12 · 88 · 159

Avoid placing labels at every geometry vertex.

Place labels around the visual midpoint of a segment or use Leaflet path-label support if already available.

When too many routes share one segment, display a shortened label:

12 · 88 · 159 · +4

Label rendering is lower priority than editing and route highlighting.

## 12. Interchange display

Interchange status is derived, not manually stored.

For each active node:

```
degree = number of active incident segments
```

Display an interchange symbol when:

```
degree > 2
```

Route endpoints of degree one should remain visible as ordinary endpoint nodes.

Degree-two nodes may remain visible in the editor but can be visually reduced or hidden in the public viewer when they are not otherwise important.

## 13. Public viewer

Create a safe read-only public viewer.

Suggested routes:

```
/
```

Public map viewer.

```
/editor
```

Editor.

Suggested APIs:

```
GET /api/public/network
GET /api/public/routes
GET /api/public/routes/{route_id}
GET /api/public/segments/{segment_id}
```

Public endpoints must expose only published network data and route metadata.

They must not expose:

- edit-session tokens;
- write operations;
- internal debugging information;
- filesystem paths;
- unsafe metadata;
- mutation endpoints.

Editor mutations must not be protected only by hidden UI controls.

For the simplest first deployment, support one editor password or authenticated editor user.

The viewer must support:

- pan and zoom over an OSM basemap;
- click segment to list bus routes using it;
- search or select a bus route;
- highlight the selected route;
- optionally select a directional variant;
- clear route selection;
- fit map to selected route;
- display interchange nodes;
- optionally show route labels;
- route selection through URL query parameters.

Example:

```
/?route=73
```

Optional:

```
/?route=73&direction=northbound
```

## 14. OSM basemap

Keep Leaflet unless there is a compelling technical reason to replace it.

The existing project already vendors Leaflet and uses it successfully.

Use an OSM-compatible tile provider.

Do not depend on the public OpenStreetMap tile service for unrestricted production traffic.

Make the tile URL, attribution, and maximum zoom configurable.

## 15. Edit-session integration

Retain the replay-based edit-session model.

Add operation types as needed:

```
create_route
update_route
delete_route
create_route_direction
update_route_direction
draw_route_path
```

```
create_path_segment
assign_route_to_segment
remove_route_from_segment
set_segment_direction
```

Where practical, combine repetitive route-membership clicks into one operation group so that one paint session can be undone coherently.

The existing project derives staged state by replaying operations against a saved snapshot. Keep that model because it makes Undo and atomic Save reliable.

A staged response should include:

```
network
routes
routeDirections
routeMemberships
operationSummary
changeSummary
canUndo
canCommit
isStale
```

## 16. Atomic save requirements

Extend the current atomic commit protocol.

Before commit validate:

- every final segment references valid nodes;
- segment geometry starts and ends at the referenced node coordinates;
- segments contain at least two distinct positions;
- route memberships reference valid routes, route directions, and segments;
- route-direction endpoints reference valid nodes;
- membership traversal values are valid;
- no membership duplicates exist;
- segment direction restrictions are not contradicted;
- route codes remain unique.

Within one transaction:

1. verify the base network revision;
2. delete replaced or deleted route memberships;
3. delete replaced or deleted segments and affected R-tree rows;
4. insert added nodes;
5. insert added segments;
6. update R-tree rows;
7. map temporary node and segment IDs to permanent IDs;
8. insert or update routes;

9. insert or update route directions;
10. insert route-direction segment memberships;
11. remove genuinely orphaned nodes where appropriate;
12. commit once.

Any failure must roll back the complete transaction.

## 17. API additions

Suggested endpoints.

### Routes

```
GET    /api/routes
POST   /api/edit-sessions/{token}/routes
PATCH /api/edit-sessions/{token}/routes/{route_id}
DELETE /api/edit-sessions/{token}/routes/{route_id}
```

### Route directions

```
POST    /api/edit-sessions/{token}/routes/{route_id}/directions
PATCH  /api/edit-sessions/{token}/route-directions/{direction_id}
DELETE  /api/edit-sessions/{token}/route-directions/{direction_id}
```

### New segments and route drawing

```
POST /api/edit-sessions/{token}/path-segments
POST /api/edit-sessions/{token}/route-drawings
```

A route-drawing request may contain:

```
{
  "routeDirectionId": 123,
  "startTarget": {
    "type": "new_point",
    "longitude": -0.12,
    "latitude": 51.51
  },
  "coordinates": [
    [-0.12, 51.51],
    [-0.121, 51.512],
    [-0.123, 51.514]
  ],
  "endTarget": {
    "type": "existing_segment",
    "pathSegmentId": 456,
    "longitude": -0.123,
```



```
    "latitude": 51.514
  }
}
```

The backend remains authoritative.

The browser sends user intent and raw clicked coordinates. The backend performs:

- projection onto selected segments;
- endpoint tolerance checks;
- segment splitting;
- node reuse;
- topology validation;
- route membership inheritance.

## Route memberships

```
PUT    /api/edit-sessions/{token}/route-directions/{direction_id}/segments/
{segment_id}
DELETE /api/edit-sessions/{token}/route-directions/{direction_id}/segments/
{segment_id}
```

Input:

```
{
  "traversal": "start_to_end"
}
```

## Existing edit endpoints

Retain and adapt:

```
POST    /api/edit-sessions
POST    /api/edit-sessions/{token}/path-segments/{id}/split
DELETE  /api/edit-sessions/{token}/path-segments/{id}
DELETE  /api/edit-sessions/{token}/junctions/{id}
POST    /api/edit-sessions/{token}/junctions/{id}/move
POST    /api/edit-sessions/{token}/undo
POST    /api/edit-sessions/{token}/commit
DELETE  /api/edit-sessions/{token}
```

# 18. Implementation phases

## Phase 1: Repository audit and safe extraction

- run the existing test suite;
- document current entry points;

- identify reusable modules;
- identify hiking-only modules;
- preserve existing behaviour before modifications;
- decide whether to fork or add a bus application mode;
- add migrations rather than replacing the database manually.

Deliverable:

docs/bus-map-adaptation.md

Include module reuse decisions and migration approach.

## Phase 2: Bus-route persistence

- add route tables;
- add route-direction tables;
- add membership table;
- add segment direction mode;
- add migrations;
- add repository functions;
- add revision hashing for new entities;
- return bus data through the network API;
- add unit and repository tests.

Completion criteria:

- routes survive restart;
- memberships survive restart;
- selecting a route through an API query returns its segments;
- splitting a segment copies route memberships correctly.

## Phase 3: Basic route viewer

- add route list;
- click segment to show routes;
- select route to highlight all tagged segments;
- support route-direction selection;
- derive direction names from endpoints;
- show interchanges where degree is greater than two;
- add URL route selection.

Completion criteria:

- viewer is read-only;
- route selection highlights every matching segment;
- clicking a shared segment lists all attached routes.

## Phase 4: Route-management editor

- create route;
- create directional variants;

- set start and end nodes;
- derive direction names;
- assign and remove memberships from existing segments;
- set traversal;
- add route brush mode;
- add Undo and atomic Save coverage.

Completion criteria:

- a route can be built entirely from existing network segments;
- route memberships can be undone;
- no viewer mutation endpoint is exposed.

## **Phase 5: Standalone segment drawing**

Build on the existing planned new-segment workflow, which already anticipates start junctions, shape points, end junctions, and keyboard controls.

Extend it to support:

- start at empty map position;
- start at existing node;
- start on an existing segment;
- finish at empty map position;
- finish at existing node;
- finish on existing segment;
- create new endpoint nodes;
- split target segments;
- add shape points;
- duplicate warnings;
- staged preview;
- Undo, Cancel, Save.

Completion criteria:

- a segment can be drawn from nowhere;
- both endpoints are nodes;
- clicking a segment creates or reuses the correct connection node;
- no duplicate overlapping segment is created accidentally.

## **Phase 6: Draw complete bus route**

- select active route direction;
- start from anywhere;
- create geometry;
- connect to existing network;
- reuse existing segments;
- automatically add memberships;
- continue drawing from the latest node;
- finish the route at a new or existing node.

Completion criteria:

- a new route can be created without pre-existing nodes;
- one drawing workflow can combine new and reused segments;
- every completed segment receives the active route membership;
- splitting shared segments preserves all existing memberships.

### **Phase 7: Public/editor separation**

- add editor authentication;
- separate public and editor API routes;
- audit all mutation endpoints;
- add CSRF protection where relevant;
- add secure cookie settings;
- add production configuration;
- add a read-only public deployment mode.

Completion criteria:

- anonymous users cannot mutate data;
- editor requests fail without authentication;
- public viewer contains no editing controls;
- read-only API responses expose only required data.

### **Phase 8: Labels and usability refinement**

- optional route labels;
- route search;
- route badges;
- route fit-to-bounds;
- better selected-route styling;
- editor keyboard shortcuts;
- route continuity warnings;
- mobile/touch review.

## **19. Tests**

### **Data-model tests**

- unique route code;
- route direction belongs to route;
- route-direction endpoints reference valid nodes;
- membership references valid segment;
- duplicate memberships rejected;
- traversal values validated;
- endpoint-derived direction naming;
- segment defaults to bidirectional.

### **Split inheritance tests**

- no-route segment split;
- segment with one route membership;

- segment with several memberships;
- forward traversal copied correctly;
- reverse traversal copied correctly;
- both-direction traversal copied correctly;
- segment-level direction mode preserved;
- Undo restores the source segment and memberships;
- Save persists both child memberships atomically.

### **Drawing tests**

- start on empty map;
- start at existing node;
- start on existing segment;
- add several shape points;
- finish on empty map;
- finish at existing node;
- finish on existing segment;
- reject zero-length segment;
- reject duplicate geometry;
- endpoint-near split reuses node;
- interior segment click creates split node;
- route membership added automatically;
- drawing Undo restores topology and memberships.

### **Route-viewer tests**

- clicking segment lists one route;
- clicking shared segment lists several routes;
- selecting route highlights all memberships;
- selecting route direction highlights directional subset;
- disconnected route memberships still render;
- no selected route clears highlight;
- route query parameter loads correctly;
- interchange display follows node degree.

### **Security tests**

- anonymous viewer GET succeeds;
- anonymous mutation fails;
- authenticated editor mutation succeeds;
- public endpoint cannot create an edit session;
- invalid edit-session token fails;
- stale session fails commit;
- failed commit leaves graph and memberships unchanged.

## **20. Non-goals for the first release**

Do not implement yet:

- live bus locations;
- actual bus stops;
- timetables;

- TfL API integration;
- automatic OSM road snapping;
- automatic route import;
- route ordering or full pathfinding;
- route continuity enforcement;
- turn-by-turn directions;
- collaborative editing;
- multi-user conflict resolution beyond stale-session rejection;
- vector-tile generation;
- offline mode;
- automatic labels at every zoom;
- automatic inference of route direction from all geometry.

## 21. Coding principles

- Prefer modifying and extending proven existing modules over introducing a new framework.
- Keep Flask, Leaflet, SQLite, and the existing replay model unless repository inspection reveals a concrete blocker.
- Keep topology decisions on the backend.
- Keep browser state transient and replaceable from the complete server-derived session response.
- Use database migrations.
- Preserve existing tests where shared functionality remains.
- Add focused tests before large UI changes.
- Avoid broad renaming refactors during functional work.
- Keep route membership normalized in relational tables.
- Treat route highlighting as a query over memberships, not as stored duplicate geometry.
- Keep segment geometry independent from route identity.
- Preserve atomicity across topology and route-membership changes.

## 22. Expected final result

The resulting application should allow this complete workflow:

```
Create Route 73
Create one directional variant
Click empty map to create the first node
Draw shape points
Click an existing segment
Split that segment and connect to it
Reuse several existing segments
Leave the existing network by drawing a new segment
Finish at a new endpoint node
Save
```

The public viewer should then allow this workflow:

```
Open map
Select Route 73
```

See every segment assigned to Route 73 highlighted  
Select one segment  
See every bus route that uses it  
Clear the selection

The implementation should feel like a simplified evolution of RouteManagement, not a new application built beside it.